

Thesis Proposal:

Automated External Attack Surface Mapping (ASM)

Student: Arvin Allahbakhsh, arvina@chalmers.se

Company: Nordic Defender, <https://nordicdefender.com/>

Company Supervisor: Mahsima Jalooli, mahsima.jalooli@nordicdefender.com

Preferred Chalmers Supervisor: Sakib Sisteek (previous contact established)

Language: English, The host company operates internationally and the supervisor prefers communication in English.

Project Description

Field: Cybersecurity - Attack Surface Management (ASM)

Background

In the current era of “Vibecoding” and AI-assisted development, the speed of software production has increased exponentially. While this allows both professionals and hobbyists to deploy applications rapidly, it often leads to a “security debt” where cloud infrastructures, subdomains, and APIs are published without proper security audits. As these digital footprints grow, forgotten or misconfigured assets become major entry points for attackers. Attack Surface Mapping (ASM) is the vital process of discovering and monitoring these external assets from an attacker’s perspective.

Problem Statement

Manual identification of these assets is time-consuming and prone to human error. Existing tools often lack the speed, the specific logic needed to filter out noise from actual threats or are too complex for non-security professionals. Nordic Defender, as a cybersecurity firm, sees a constant need for efficient ways to help clients find these “open windows” before they are exploited by hackers.

Objectives & Tasks

The goal of this thesis is to develop a standalone software module that automates the discovery and analysis of a company’s external assets using public Open Source Intelligence (OSINT) data. The tool should be intuitive enough for a “vibecoder” to verify their deployment, yet robust enough for a professional security analyst.

Key Tasks include:

- **Asset Discovery:** Implementing logic to scan public DNS records, certificate transparency, logs and IP ranges to find subdomains and hidden servers.

- **Service Fingerprinting:** Developing a scanning engine to identify what software and versions are running on discovered ports to detect vulnerabilities (e.g, identifying an outdated Apache server).
- **Technical Language Evaluation (Python vs GoLang):** A core part of the start of the project will be to compare GoLang (known for its concurrency and speed in networking) and Python (known for its extensive security libraries). I will evaluate which language is most efficient for building a high-performance ASM engine.
- **Data Modeling & UI/UX:** Structuring the discovered data in a way that allows for easy tracking of changes over time (e.g, using a PostgreSQL database). If time permits, a clean and simple UI will be developed to make the security data accessible to non-experts.

Preliminary Timeline

- **Phase 1: Research & Setup:** Comparative evaluation of Python vs GoLang for network scanning. Setting up the sandboxed Docker environment and implementing basic DNS discovery logic.
- **Phase 2: Backend Development:** Core development of the scanning engine, focusing on Asset Discovery and Service Fingerprinting algorithms.
- **Phase 3: Data Modeling:** Structuring the discovered data within a PostgreSQL database.
- **Phase 4 (If time permits): UI development:** Developing a simplified dashboard/UI
- **Phase 5: Analysis & Final Documentation:** Technical evaluation of the results, comparing the implemented solutions, and finalizing the thesis report. *Note: Documentation and analysis will be performed continuously throughout all phases to ensure academic quality.*

Relevant Coursework

The project builds upon the following courses completed at Chalmers University of Technology:

- **Computer Security:** Understanding threats, vulnerabilities, and defensive strategies.
- **Computer Communication:** Deep dive into network protocols (TCP/IP, DNS, HTTP) which is essential for scanning.
- **Distributed Systems:** Handling data and communication across different nodes.
- **Operating Systems:** Knowledge of the network stack and process management.
- **Databases:** Experience in structuring and querying relational data.
- **OOP & OOD:** Applying object-oriented design with high cohesion and low coupling.

Implementation & Independence

The project will be carried out by the student with technical guidance and industry best practices provided by Nordic Defender. While the student is responsible for the coding and execution, Nordic Defender will provide guidelines, tips, and expert feedback to ensure that tool's relevance to the cybersecurity market.

Since Nordic Defender handles sensitive data, this project is designed to use only public data. This ensures that the final report remains a public document without compromising company or client confidentiality. All development and testing will be conducted in a sandboxed Docker environment to ensure security and maintainability.